

Strata Unleashed

Model selection, verified downloads and per-machine inference tuning

Engineering note · release 0.1.38-r3 · 3 October 2026

This note describes the Unleashed fork's installation and tuning workflow. It is not a peer-reviewed paper or a replacement for the original Strata paper. Engine architecture and upstream contributions remain attributed to Strata and its dependencies.

1. Purpose and scope

A large mixture-of-experts model can be difficult to install even when the inference engine is already capable of running it. Users must choose a compatible quantization, find every shard, estimate memory and disk requirements, prepare auxiliary data, and select runtime settings. Unleashed brings these steps into one guided interface on Windows and Linux.

The workflow targets the Qwen3.8-Flash-Next qwen4exp family on an x86-64 CPU and one NVIDIA GPU. It checks the actual machine at setup time. Paths, GPU index, RAM, VRAM, drive type and measured settings are not copied from the development computer. Unsupported platforms receive a diagnostic rather than a claim of universal compatibility. Upstream AMD, multi-GPU and specialized low-RAM workflows are separate from this guided tuner.

The app is distributed as verified release archives. Linux includes an Ada engine; Windows supplies source and build scripts. Python packages are installed locally. Drivers and compiler/CUDA prerequisites remain explicit requirements, and a source build is requested when the recorded binary architecture does not match the machine.

2. Catalog and download integrity

The bundled catalog contains 22 complete model variants across ISTA general, ISTA Coder, Atomic Chat, HuiHui and Unsloth. It pins each publisher's immutable revision and records every shard's size and SHA256. The initial catalog inspection read 170 shard headers. Complete-file hashes are checked when weights are downloaded; inspecting a header alone is not a substitute for that verification.

The catalog groups files by shard family and rejects incomplete sets. It excludes MTP heads, vision projectors and imatrix files from the list of standalone language models. Compatibility is based on actual tensor types and layouts, not the quant label in a filename. For example, a mixed model labeled Q2_K may contain supported expert types even though the engine does not generally implement Q2_K experts.

Nineteen catalog variants passed header/layout checks. HuiHui BF16, Unsloth BF16 and Unsloth UD-IQ1_S were blocked by unsupported expert encodings. This does not mean all other variants have passed full-model inference or quality evaluation; the recorded full-model benchmark used Atomic Q5.

Downloads use immutable URLs and temporary .part files. Resume checks the server's byte range; a server that ignores the range triggers a restart rather than corrupt concatenation. A completed shard is renamed only after its size and SHA256 match. A process lock prevents two setup instances from writing the same partial file.

Recommendation preferences follow the requested product behavior: ISTA for compact/general and coding use, Atomic for larger choices, and HuiHui for uncensored/ablated use. Unsloth remains an alternative. These preferences are not an experimentally established quality ordering.

3. Machine and storage assessment

The scanner reads the operating system, CPU, installed and available RAM, free disk, NVIDIA device/driver information, current free VRAM, build tools and engine compatibility. A model that exceeds an estimate receives a specific reason, such as insufficient currently available RAM or a missing compiler. Users can change KV precision or reserved VRAM and compare the results.

The current resident-expert RAM estimate is expert bytes plus twice dense-weight bytes plus 6 GiB overhead. Context beyond 16,384 tokens adds 64 KiB per extra token. Disk estimates include remaining downloads, converted pack bytes and 2 GiB working headroom. The PLE table is mapped from disk rather than treated as a permanently resident RAM allocation.

The advisory startup VRAM estimate includes dense weights, a cache allowance for the selected precision across 12 full-attention layers with two KV heads and 256 values per head, and 2 GiB workspace. These estimates can reject marginal setups that manual configuration or another engine might run. They do not prove a universal lower bound.

Drive identification is performed on the volume actually containing the app or selected model. Linux follows the mounted block device to its parent transport; Windows queries the partition's disk bus. NVMe is reported only when identified as such. SATA, USB, rotational and unknown storage receive an experience-may-vary notice.

A bounded sequential sample reads up to 32 MiB from an existing file using Linux O_DIRECT or Windows unbuffered reads. No weight files or disk-test files are written. If the operation is unsupported, speed remains unavailable rather than being replaced with a cached-RAM measurement. A result below 500 MiB/s produces an advisory warning even for NVMe. This threshold is a usability heuristic, not a measured inference boundary. Controller caching, load and short sample duration limit interpretation; random-I/O latency is not measured.

4. Procedural tuning

The measured memory limit is total device memory minus the user's additional reserve and automatic safety headroom. Safety headroom is the greater of 512 MiB or 2 percent of total VRAM. Sampling includes memory already used by the desktop and other processes. Fit trials adjust the engine's internal allocation allowance, and an out-of-memory failure raises the retained backoff floor.

The KV default is FP16/FP16 for nominal 16 GiB or larger cards, FP16/Q8 for 12–16 GiB, Q8/Q8 for 8–12 GiB, and Q8/Q6 below 8 GiB. Driver-reported capacity may be up to 64 MiB below a nominal tier. Users may explicitly override this with --kv; the tuner does not change cache precision automatically.

The first objective is generation throughput on two 512-input/512-output workloads. The search fixes K/V precision to the GPU-capacity default (or an explicit override), and considers optional MTP settings, CPU workers, uncached-expert PCIe share and other existing runtime controls. Fresh incumbent controls and repeated comparisons reduce the risk that temperature, clocks or link-state changes make an old measurement dominate subsequent choices.

The second objective is prompt processing, tested on a 261,624-input/512-output workload. Every trial, including short decode tests, allocates native 262,144-token context. No context reduction or RoPE extension is used. A candidate must retain at least 90 percent of the measured generation baseline. Final fresh-engine checks validate the chosen settings before publishing best-config.json. If no candidate passes, the tuner does not silently label a failed run optimal.

This procedure searches a finite set of supported controls and nearby refinements. It is not a proof of a global optimum and does not predict the same speed on another machine. The saved fingerprint includes model, engine, tuning code, hardware and relevant configuration so changed conditions can trigger a new search.

5. Terminal inference testing

The terminal path is intentionally small: user and assistant turns, the model's ordinary chat framing, and direct access to the prepared engine. It supplies no system role, tools or agent instructions. Template thinking instructions are disabled, and the rendered prompt is checked for a system turn before loading or generating.

This is zero system prompt, not zero formatting: the model still receives its expected user/assistant delimiters. An incompatible template is rejected rather than silently rewritten. A conversation can be reset with /new. The reply limit is adjustable with /tokens, and /quit closes the engine.

Displayed throughput counts generated token IDs and uses engine decode timing when available. It does not call HTTP chunks tokens. Total elapsed time is reported separately. Tiny replies, cold starts and interactive conversations are not directly comparable to the fixed 512-token benchmark.

6. Recorded evidence

The original Unleashed Q5 validation used an NVIDIA RTX 5000 Ada Generation Laptop GPU with 16,376 MiB VRAM, CUDA 13.1, driver 595.91.07 and approximately 128 GB installed RAM. The model was the 33-shard Atomic Q5 release. The validated binary hash and full result summary are retained in docs/UNLEASHED-VALIDATION.json.

For 512 input and 512 output tokens, the selected configuration's median generation speed was 51.64 tokens/s. For 8,192 input and 512 output tokens, median prompt processing was 1,644.02 tokens/s. Observed peak VRAM was 15,844 MiB. The selected prefill configuration reduced short-workload generation by 3.24 percent relative to the final decode reference, within the 10 percent acceptance rule.

A separate 2,048 MiB extra-reserve check peaked at 13,794 MiB, below its 13,816 MiB limit. That was a reserve-behavior validation, not an independent complete optimization sweep for that reserve.

The 0.1.38-r3 revision adds Q8/Q6 cache parsing and fixes native-context tuning policy. The earlier throughput and chat results above used the previous engine and reduced context; they do not validate native-context throughput. Regression coverage includes fit estimates across simulated GPU/OS combinations, archive extraction, download resume and corruption handling, no-system prompt rendering, storage notices and read-only direct-I/O behavior. A real terminal request against the local Q5 model returned the requested answer. Native Windows execution is still unverified; simulated Windows tests are explicitly labeled as such.

The native-context revision separately passed a Q5 FP16/FP16 run without MTP: 261,624 input tokens, 512 generated tokens and eight internal margin tokens within a 262,144-token allocation. A single sample measured approximately 451 prefill tokens/s and 25.7 decode tokens/s near the context limit, with about 95 percent total VRAM usage. This is a capacity/inference validation, not a new complete optimization sweep. See docs/UNLEASHED-NATIVE-VALIDATION.json for the engine hash and exact measurements.

7. Reproduction and limitations

Use the model picker to create a configuration on the target machine; do not copy the development PC's absolute paths or expert-cache sizes. Preserve the resulting measurements and note background GPU activity, drive location, context, reserve, model revision and engine hash. Treat published benchmark values as observations under those recorded conditions.

The tested catalog does not cover arbitrary architectures or every possible GGUF encoding. New publisher revisions require fresh header checks and download hashes. Header compatibility is weaker evidence than numerical parity, successful generation, or a quality evaluation. Windows scripts require native testing before a tested Windows binary can be claimed.

The installer checks archive integrity and confines application/model data to its destination. It does not overwrite an unrelated directory or update an existing checkout without the user's deliberate action. Local

credentials, weight files, caches and transcripts are not release sources.

References

1. Niko1221/Strata: <https://github.com/Niko1221/Strata> and the retained original paper, docs/paper/Strata-Paper.pdf.
2. ISTA general and Coder model repositories:
<https://huggingface.co/ISTA-DASLab/Qwen3.8-Flash-Next-GSQ-RCO-GGUF> and
<https://huggingface.co/ISTA-DASLab/Qwen3.8-Flash-Next-GSQ-RCO-Coder-GGUF>.
3. Atomic Chat: <https://huggingface.co/AtomicChat/Qwen3.8-Flash-Next-GGUF>.
4. HuiHui: <https://huggingface.co/huihui-ai/Huihui-Qwen3.8-Flash-Next-abliterated-GGUF>.
5. Unsloth: <https://huggingface.co/unsloth/Qwen3.8-Flash-Next-GGUF>.
6. Unleashed catalog, validation and source: <https://github.com/averyjohnkucan-arch/StrataUnleashed>.

Original model and dependency licenses apply. The original Strata paper and diagrams remain credited to their upstream authors.